
Programming with Python

BASIC CONCEPTS 3

Lesson Objectives

- Modules
- Packages
- Try...except...finally

Section 1

Modules

Modules

➤ What are Modules?

Modules are files containing Python definitions and statements (ex. *name.py*)

A module's definitions can be imported into other modules by using "import *name*"

The module's name is available as a global variable value

To access a module's functions, type "*name.function()*"

Modules

- The *import* Statement

- **import <module_name>**
- **import <module_name> as <alt_name>**

- Example:

- **import numpy**
- **import numpy as np**

Modules

- The *from* Statement

- `from <module_name> import <name(s)>`
- `from <module_name> import <name> as <alt_name>`

- Example

- `from tensorflow import keras`
- `from tensorflow import keras as K`

Modules

- Executing **Modules**

- `python name.py <arguments>`
 - Runs code as if it was imported
 - Setting `_name_ == "_main_"` the file can be used as a script and an importable module

Modules

- Advantages to **modularizing** code in a large application
 - **Simplicity**
 - **Maintainability**
 - **Reusability**
 - **Scoping**

Section 2

Packages

Packages

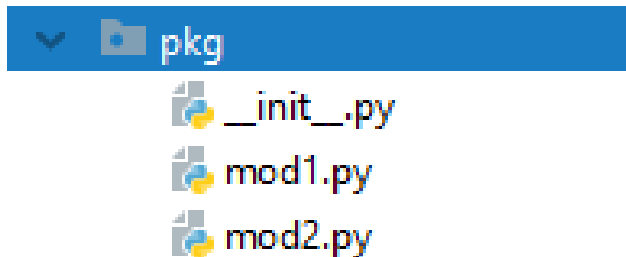
- **Packages** allow for a hierarchical structuring of the module namespace using **dot notation**.
- Creating a **package** is quite straightforward, since it makes use of the operating system's inherent hierarchical file structure



Packages

- **Package Initialization**

- If a file named `__init__.py` is present in a package directory, it is invoked when the package or a module in the package is imported.
- This can be used for execution of package initialization code, such as initialization of package-level data



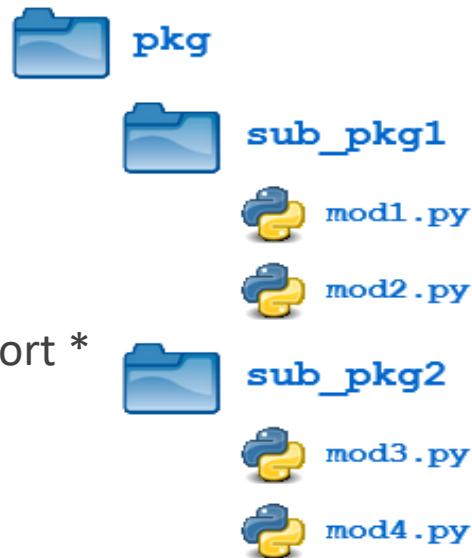
Packages

- **Importing * From a Package**

- `from <package_name> import *`

- **Subpackages**

- `from <package_name_1>. <package_name_1.1> import *`



Modules - Packages

□ Python searches modules in the following manner:

1. Searches in current directory
2. If the module is not found then searches each directory in the shell variable PYTHONPATH
3. If all else fails, python checks the default path which is the installation location of the python

Packages Management - PIP

□ What is pip?

- Pip is a package-management system used to install and manage software packages written in python
- Pip install <package-name>

Packages Management - PIP

- ❑ List all packages installed
 - Pip freeze
 - `help("modules")`

Section 3

Try...except...finally

Try...except...finally

- Exceptions versus Syntax Errors
- Raising an Exception
- The ***AssertionError*** Exception
- Handling Exceptions
- The ***else*** Clause



Try...except...finally

- Exceptions versus Syntax Errors
 - **Syntax errors** occur when the parser detects an incorrect statement.

```
print( 0 / 0 ))
```

```
File "<ipython-input-1-c3931f671051>", line 1  
print( 0 / 0 ))  
      ^
```

Try...except...finally

- Raising an Exception

- We can use ***raise*** to throw an exception if a condition occurs.

```
x = 10
if x > 5:
    raise Exception('x should not exceed 5. The value of x was: {}'.format(x))
```

Try...except...finally

- The ***AssertionError*** Exception

- We ***assert*** that a certain condition is met. If this condition turns out to be ***True***, then that is excellent! The program can continue. If the condition turns out to be ***False***, you can have the program throw an ***AssertionError*** exception.

```
import sys
assert ('linux' in sys.platform), "This code runs on Linux only."
```

Try...except...finally

- The **try** block lets you test a block of code for errors.
- The **except** block lets you handle the error.
- The **finally** block lets you execute code, regardless of the result of the try- and except blocks.

Try...except...finally

- The ***try*** and ***except*** Block

- The *try* and *except* block in Python is used to catch and handle exceptions.

try:



Run this code

except:



Execute this code when
there is an exception

Try...except...finally

```
def linux_interaction(os_name):  
    assert (os_name in sys.platform), "Function can only run on Linux systems."  
    print('Doing something.')
```

```
try:  
    linux_interaction('windows')  
except:  
    print('Linux function was not executed')
```

Linux function was not executed

Try...except...finally

- The

try:



Run this code

except:



Execute this code when
there is an exception

else:



No exceptions? Run this
code.

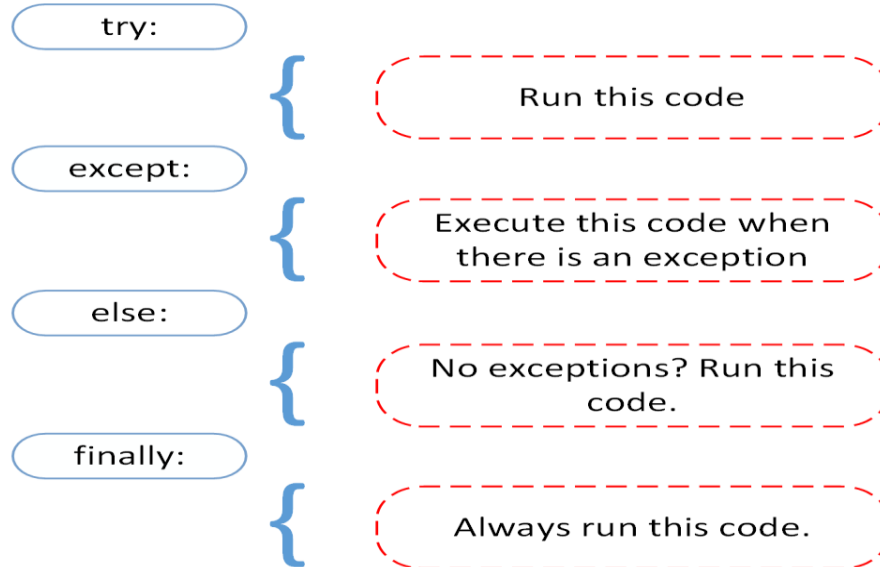
Try...except...finally

- The ***else*** Clause

```
try:
    linux_interaction('linux')
except AssertionError as error:
    print(error)
else:
    print('Executing the else clause.')
```

Try...except...finally

- Cl



Try...except...finally

- Cleaning Up After Using *finally*

```
try:
    linux_interaction('windows')
except AssertionError as error:
    print(error)
else:
    try:
        with open('file.log') as file:
            read_data = file.read()
    except FileNotFoundError as fnf_error:
        print(fnf_error)
finally:
    print('Cleaning up, irrespective of any exceptions.')
```

Try...except...finally

- Customizing Exception Classes

```
class SalaryNotInRangeError(Exception):  
    def __init__(self, salary, message="Salary is not in (5000, 15000) range"):  
        self.salary = salary  
        self.message = message  
        super().__init__(self.message)  
  
def salary_validate(salary):  
    if not 5000 < salary < 15000:  
        raise SalaryNotInRangeError(salary)
```

Try...except...finally

- Using Custom Exception Classes

```
try:
    salary = int(input("Enter salary amount: "))
    salary_validate(salary)
except SalaryNotInRangeError as sni:
    print(sni.message)
```

Thank you
